🛠️ Phân Tích Siêu Chi Tiết: Cách Tạo Extension VEO Automation

> ***Extension:*** *VEO Automation - Auto VEO on Google Flow*
> ***Developer:*** *Truong Nguyen (kylenguyen.me) | Hà Nội, Việt Nam* VN
> ***Version:*** *2.3.6* | ***Users:*** *10,000+* | ***Rating:*** ⭐ *4.7/5*

📸 Giao Diện Thực Tế

```
![Screenshot 1 - Main UI với Side Panel hiển thị các mode: Text to Video, Frame to
Video, Ingredients to Video, Text to Image, Image to Image, Last Image To Image]
(C:\Users\intel\.gemini\antigravity\brain\830c6321-dd06-4812-adc3-
344869cea7c5\veo_ss2_1772203898902.png)
<!-- slide -->
![Screenshot 2 - Queue Management Panel: Danh sách prompts với Duration per Prompt,
progress tracking cho từng item trong hàng đợi]
(C:\Users\intel\.gemini\antigravity\brain\830c6321-dd06-4812-adc3-
344869cea7c5\veo_ss3_1772203931649.png)
<!-- slide -->
![Screenshot 3 - Tổng quan trang Chrome Web Store]
(C:\Users\intel\.gemini\antigravity\brain\830c6321-dd06-4812-adc3-
344869cea7c5\full_page_veo_automation_1772202940712.png)
```

I. 🏗️ Kiến Trúc Tổng Thể (Manifest V3)

Extension này chạy trên **Chrome Extension Manifest V3 (MV3)** — chuẩn mới nhất của Google Chrome kể từ 2023.

```
graph TB
    subgraph "Chrome Extension (711 KiB)"
        SP[🖥️ Side Panel UI\nReact/Vanilla JS\nHTMLCSS]
        SW[⚙️ Service Worker\nBackground Logic\nQueue Manager]
        CS[💉 Content Script\nDOM Automation\nlabs.google.com]
        MP[📨 Message Passing\nchrome.runtime API]
    end

    subgraph "Target Website"
```

```
        GF[🌐 Google Flow\nlabs.google.com/fx/tools/flow]
        DOM[📄 DOM Elements\nInput, Buttons\nVideo Players]
    end

    subgraph "Chrome APIs"
        DL[📥 chrome.downloads]
        TABS[📑 chrome.tabs]
        ST[💾 chrome.storage]
        I18N[🌐 chrome.i18n]
    end

    SP <-->|Messages| SW
    SW <-->|chrome.tabs.sendMessage| CS
    CS <-->|Manipulate| DOM
    DOM -->|From| GF
    SW -->|Trigger| DL
    SW -->|Monitor| TABS
    SW <-->|Persist Queue| ST
    SP -->|Display lang| I18N
```

II. 📁 Cấu Trúc File Extension

```
veo-automation/
│
├── 📄 manifest.json              ← "Giấy khai sinh" của extension
│
├── 🖥️ side-panel/
│   ├── panel.html                ← Giao diện Side Panel
│   ├── panel.js                  ← Logic UI: tabs, form inputs
│   └── panel.css                 ← Styling (dark theme)
│
├── ⚙️ background/
│   └── service-worker.js         ← Queue engine, orchestrator
│
├── 🚀 content-scripts/
│   ├── flow-automator.js         ← DOM manipulation trên Google Flow
│   ├── image-handler.js          ← Upload/handle images
│   └── video-downloader.js       ← Detect & download videos
│
├── 🌐 _locales/
│   ├── en/messages.json          ← English
```

```
│   ├── vi/messages.json          ← Tiếng Việt
│   ├── zh/messages.json          ← 中文
│   ├── ko/messages.json          ← 한국어
│   ├── ja/messages.json          ← 日本語
│   └── es/messages.json          ← Español
│
├── 🖼️ icons/
│   ├── icon-16.png
│   ├── icon-32.png
│   ├── icon-48.png
│   └── icon-128.png              ← Con chuột hamster/beaver cute
│
└── 📦 lib/
    ├── queue-manager.js          ← Smart Queue implementation
    └── utils.js                  ← Helper functions
```

III. 📄 manifest.json — "Giấy Khai Sinh" Extension

```json
{
  "manifest_version": 3,
  "name": "__MSG_extensionName__",
  "version": "2.3.6",
  "description": "__MSG_extensionDescription__",
  "default_locale": "en",

  "permissions": [
    "sidePanel",
    "storage",
    "downloads",
    "scripting",
    "tabs",
    "activeTab"
  ],

  "host_permissions": [
    "https://labs.google.com/fx/tools/flow/*",
    "https://aisandbox-pa.googleapis.com/*"
  ],

  "background": {
    "service_worker": "background/service-worker.js",
```

```
    "type": "module"
  },

  "content_scripts": [
    {
      "matches": ["https://labs.google.com/fx/tools/flow/*"],
      "js": ["content-scripts/flow-automator.js"],
      "run_at": "document_idle"
    }
  ],

  "side_panel": {
    "default_path": "side-panel/panel.html"
  },

  "action": {
    "default_icon": "icons/icon-32.png",
    "default_title": "VEO Automation"
  },

  "icons": {
    "16": "icons/icon-16.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  }
}
```

> *[!IMPORTANT]* `host_permissions` *chỉ cho phép extension chạy trên đúng domain Google Flow. Đây là yếu tố bảo mật quan trọng để được Google Web Store chấp thuận.*

IV. ⚙️ Service Worker — Bộ Não Điều Phối

`background/service-worker.js` — không có DOM, chạy như một "server nhỏ" trong Chrome.

```
// === KHỞI TẠO ===
chrome.runtime.onInstalled.addListener(() => {
  // Mở Side Panel khi click icon extension
  chrome.sidePanel.setPanelBehavior({ openPanelOnActionClick: true });
});

// === QUEUE ENGINE ===
```

```
class QueueManager {
  constructor() {
    this.queue = [];       // Danh sách jobs chờ
    this.running = [];     // Jobs đang chạy (concurrency)
    this.maxConcurrent = 2; // Chạy tối đa 2 jobs song song
  }

  async add(promptJob) {
    this.queue.push(promptJob);
    await this.saveToStorage(); // Lưu persistent
    this.processNext();         // Bắt đầu xử lý
  }

  async processNext() {
    if (this.running.length >= this.maxConcurrent) return;
    if (this.queue.length === 0) return;

    const job = this.queue.shift();
    this.running.push(job);

    try {
      await this.executeJob(job);
    } finally {
      this.running = this.running.filter(j => j.id !== job.id);
      this.notifyPanel('job_completed', job);
      this.processNext(); // Chạy job tiếp theo
    }
  }

  async executeJob(job) {
    // Tìm tab Google Flow đang mở
    const [tab] = await chrome.tabs.query({
      url: 'https://labs.google.com/fx/tools/flow/*'
    });

    // Gửi lệnh xuống Content Script để thực hiện
    const result = await chrome.tabs.sendMessage(tab.id, {
      action: 'CREATE_AND_RUN_PROMPT',
      payload: {
        prompt: job.prompt,
        model: job.model,       // "veo-3.1-fast" | "veo-2-quality"
        aspectRatio: job.ratio, // "16:9" | "9:16" | "1:1"
        duration: job.duration, // 5s | 8s
        referenceImage: job.lastImage // last frame chaining!
      }
    });
```

```
    // Khi video xong → tự động download
    if (result.videoUrl) {
      await this.downloadVideo(result.videoUrl, job);
    }
  }

  async downloadVideo(url, job) {
    await chrome.downloads.download({
      url: url,
      filename: `flow/${job.projectName}/${job.index}_${job.prompt.slice(0,30)}.mp4`,
      conflictAction: 'uniquify'
    });
  }

  // Broadcast trạng thái hàng đợi lên Side Panel
  notifyPanel(event, data) {
    chrome.runtime.sendMessage({ event, data });
  }

  // Lưu queue vào chrome.storage để survive browser restart
  async saveToStorage() {
    await chrome.storage.local.set({ queue: this.queue });
  }
}

const queueManager = new QueueManager();

// Lắng nghe lệnh từ Side Panel UI
chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  if (msg.action === 'ADD_JOBS') {
    msg.jobs.forEach(job => queueManager.add(job));
    sendResponse({ status: 'queued' });
  }
  if (msg.action === 'GET_STATUS') {
    sendResponse({ queue: queueManager.queue, running: queueManager.running });
  }
  return true; // async response
});
```

V. 💉 Content Script — Tay Chân Thao Tác DOM

`content-scripts/flow-automator.js` — đây là phần **phức tạp và quan trọng nhất**.

```
// Chạy trong context của trang labs.google.com/fx/tools/flow

// Helper: chờ element xuất hiện (Google Flow render async)
function waitForElement(selector, timeout = 10000) {
  return new Promise((resolve, reject) => {
    const observer = new MutationObserver(() => {
      const el = document.querySelector(selector);
      if (el) { observer.disconnect(); resolve(el); }
    });
    observer.observe(document.body, { childList: true, subtree: true });
    setTimeout(() => { observer.disconnect(); reject('Timeout: ' + selector); },
timeout);
  });
}

// Helper: simulate typing vào React input (React dùng synthetic events)
function setReactInputValue(input, value) {
  const nativeInputValueSetter = Object.getOwnPropertyDescriptor(
    window.HTMLTextAreaElement.prototype, 'value'
  ).set;
  nativeInputValueSetter.call(input, value);
  input.dispatchEvent(new Event('input', { bubbles: true }));
  input.dispatchEvent(new Event('change', { bubbles: true }));
}

// Nhận lệnh từ Service Worker
chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  if (msg.action === 'CREATE_AND_RUN_PROMPT') {
    runAutomation(msg.payload).then(sendResponse);
    return true; // keep channel open for async
  }
});

async function runAutomation(config) {
  try {
    // BƯỚC 1: Click nút "New Project" hoặc tạo project mới
    const newBtn = await waitForElement('[data-action="new-flow"]');
    newBtn.click();
    await sleep(1500);

    // BƯỚC 2: Chọn loại generation (Text to Video, Image to Video...)
    await selectGenerationMode(config.mode);
```

```
    // BƯỚC 3: Nếu có reference image → upload
    if (config.referenceImage) {
      await uploadReferenceImage(config.referenceImage);
    }

    // BƯỚC 4: Điền prompt vào textarea
    const promptArea = await waitForElement('textarea[placeholder*="prompt"]');
    setReactInputValue(promptArea, config.prompt);
    await sleep(500);

    // BƯỚC 5: Chọn Model (Veo 3.1 Fast/Quality, Veo 2...)
    await selectModel(config.model);

    // BƯỚC 6: Chọn Aspect Ratio
    await selectAspectRatio(config.aspectRatio);

    // BƯỚC 7: Chọn Duration
    await selectDuration(config.duration);

    // BƯỚC 8: Click Generate!
    const generateBtn = await waitForElement('button[aria-label*="Generate"]');
    generateBtn.click();

    // BƯỚC 9: Chờ video/ảnh hoàn thành
    const resultUrl = await waitForGenerationComplete();

    // BƯỚC 10: Lấy URL video/ảnh cuối để chain
    const lastFrameUrl = await extractLastFrame(resultUrl);

    return { success: true, videoUrl: resultUrl, lastImage: lastFrameUrl };

  } catch (err) {
    return { success: false, error: err.message };
  }
}

async function waitForGenerationComplete() {
  return new Promise((resolve) => {
    // Poll mỗi 2 giây để check trạng thái
    const interval = setInterval(() => {
      // Tìm progress bar đã đầy hoặc video element xuất hiện
      const video = document.querySelector('video[src*="blob:"]');
      const progress = document.querySelector('[aria-valuenow="100"]');

      if (video || progress) {
```

```
        clearInterval(interval);
        resolve(video?.src || extractDownloadUrl());
      }
    }, 2000);
  });
}

function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

VI. 🖥️ Side Panel UI — Giao Diện Người Dùng

Từ screenshot thực tế, Side Panel có 2 tab chính: **Control** và **Setting**.

```
<!-- side-panel/panel.html -->
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>VEO Automation</title>
  <link rel="stylesheet" href="panel.css">
</head>
<body>
  <!-- Header -->
  <div class="header">
    <h2>VEO Automation - Auto Veo on Google Flow</h2>
    <span>Author: <a href="https://kylenguyen.me">kylenguyen.me</a></span>
    <button id="userGuide">📖 User Guide</button>
    <select id="langSelect"><!-- EN, VI, ZH, KO, JA, ES --></select>
  </div>

  <!-- Tabs: Control | Setting -->
  <div class="tabs">
    <button class="tab active" data-tab="control">⚡ Control</button>
    <button class="tab" data-tab="setting">⚙️ Setting</button>
  </div>

  <!-- Control Tab: Chọn loại Generation -->
  <div id="control" class="tab-content">
    <!-- Row 1: Video modes -->
```

```
    <button class="mode-btn active" data-mode="text-to-video">🎬 Text to
Video</button>
    <button class="mode-btn" data-mode="frame-to-video">🎞️ Frame to Video</button>
    <button class="mode-btn" data-mode="ingredients-to-video">🧩 Ingredients to
Video</button>

    <!-- Row 2: Image modes -->
    <button class="mode-btn" data-mode="text-to-image">🖼️ Text to Image</button>
    <button class="mode-btn" data-mode="image-to-image">🔄 Image to Image</button>
    <button class="mode-btn" data-mode="last-image-to-image">🔗 Last Image To
Image</button>

    <!-- Prompt Area -->
    <textarea id="promptInput" placeholder="Enter your prompts here...
Separate multiple prompts with a blank line. Last image will be processed with each
prompt."></textarea>

    <!-- Duration per prompt -->
    <div class="duration-selector">
      <label>⏱️ Duration per Prompt</label>
      <select id="duration">
        <option value="5s">5s</option>
        <option value="8s">8s</option>
      </select>
    </div>

    <!-- Queue Preview -->
    <div id="queueList">
      <!-- Dynamic: Hiển thị từng prompt trong hàng đợi -->
    </div>

    <!-- Run Button -->
    <button id="runBtn" class="primary-btn">▶ Start Automation</button>
  </div>

  <!-- Video Generation Progress -->
  <div class="progress-section">
    <h3>📊 Video Generation Progress</h3>
    <div id="progressList">
      <!-- Dynamic items: job name + progress bar -->
    </div>
  </div>

  <script src="panel.js"></script>
```

```
</body>
</html>
```

VII. 🌐 Internationalization (i18n) — 6 Ngôn Ngữ

```
// _locales/vi/messages.json (Tiếng Việt)
{
  "extensionName": {
    "message": "VEO Automation - Tự Động VEO trên Google Flow"
  },
  "textToVideo": {
    "message": "Văn bản thành Video"
  },
  "startAutomation": {
    "message": "Bắt đầu Tự Động Hóa"
  }
}
```

```
// Sử dụng trong JS:
document.getElementById('runBtn').textContent =
  chrome.i18n.getMessage('startAutomation');
```

VIII. 🔄 Luồng Dữ Liệu Hoàn Chỉnh (End-to-End Flow)

```
sequenceDiagram
    participant User
    participant SidePanel as 🖥️ Side Panel
    participant SW as ⚙️ Service Worker
    participant CS as 🚀 Content Script
    participant GF as 🌐 Google Flow DOM
    participant DL as 📥 chrome.downloads

    User->>SidePanel: Paste 10 prompts, click Run
```

```
    SidePanel->>SW: sendMessage(ADD_JOBS, jobs[])
    SW->>SW: Khởi tạo QueueManager với 10 jobs

    loop Với mỗi job (concurrency: 2)
        SW->>CS: sendMessage(CREATE_AND_RUN_PROMPT)
        CS->>GF: Click "New Project"
        CS->>GF: Set prompt text (React synthetic event)
        CS->>GF: Select Model (Veo 3.1 Fast)
        CS->>GF: Click Generate button
        CS->>GF: Poll status mỗi 2 giây...
        GF-->>CS: Video URL ready!
        CS->>CS: Extract last frame URL
        CS-->>SW: {success, videoUrl, lastImage}
        SW->>DL: chrome.downloads.download(videoUrl)
        SW->>SidePanel: notifyPanel(job_completed)
        SidePanel->>User: Update progress bar ✅
        SW->>SW: Feed lastImage vào job tiếp theo (chaining!)
    end

    SW->>SidePanel: All done! 10/10 videos completed
```

IX. 🔑 Các Chrome API Quan Trọng

API	Mục đích	Code mẫu
`chrome.sidePanel`	Mở Side Panel khi click icon	`chrome.sidePanel.setPanelBehavior({...})`
`chrome.tabs.sendMessage`	SW → CS gửi lệnh automation	`chrome.tabs.sendMessage(tabId, msg)`
`chrome.runtime.sendMessage`	CS/Panel → SW gửi dữ liệu	`chrome.runtime.sendMessage({action})`
`chrome.downloads`	Tải video/ảnh về máy tự động	`chrome.downloads.download({url, filename})`
`chrome.storage.local`	Lưu queue để không mất khi restart	`chrome.storage.local.set({queue})`
`chrome.i18n`	Dịch đa ngôn ngữ 6 thứ tiếng	`chrome.i18n.getMessage('key')`
`chrome.scripting`	Inject script động vào tab	`chrome.scripting.executeScript({...})`

`MutationObserver`	Chờ DOM render async (React)	`new MutationObserver(callback)`

X. 🎯 Kỹ Thuật Quan Trọng & Thách Thức

1. React DOM Manipulation (Khó nhất!)

Google Flow build bằng React. Việc set giá trị input bình thường **KHÔNG HOẠT ĐỘNG** với React vì React quản lý state riêng. Phải dùng:

```
// ❌ Sai — React không nhận ra
input.value = "my prompt";

// ✅ Đúng — Trick hack React synthetic events
const nativeSetter = Object.getOwnPropertyDescriptor(
  HTMLTextAreaElement.prototype, 'value'
).set;
nativeSetter.call(input, "my prompt");
input.dispatchEvent(new Event('input', { bubbles: true }));
```

2. CSS Selector Maintenance (Thách thức liên tục)

Google Flow thường xuyên update UI, thay đổi class names. Developer phải:

- Dùng **data-attribute selectors** thay vì class names
- Dùng **aria-label** selectors (stable hơn)
- Viết **fallback selectors** dự phòng
- Update extension thường xuyên (v2.3.6 = đã update 236+ lần!)

3. Async Timing & Race Conditions

```
// Dùng MutationObserver thay vì setTimeout cố định
// Tránh: await sleep(3000) — không đáng tin cậy
// Nên dùng: await waitForElement('[data-video-ready]')
```

4. Video Download URL Extraction

```
// Google Flow dùng blob URLs hoặc signed GCS URLs
// Phải intercept XHR/fetch response để lấy URL thật
const originalFetch = window.fetch;
window.fetch = async (...args) => {
  const response = await originalFetch(...args);
  if (args[0].includes('download') || args[0].includes('video')) {
    captureVideoUrl(args[0]);
  }
  return response;
};
```

5. Concurrency Control

```
// Xử lý 2 jobs song song nhưng không quá tải Google Flow
const semaphore = new Semaphore(2); // maxConcurrent = 2
await semaphore.acquire();
try { await runJob(job); }
finally { semaphore.release(); }
```

XI. 🛡️ Vượt Qua Chrome Web Store Review

Để được Google chấp thuận đăng lên Web Store, developer phải:

Yêu cầu	Cách thực hiện
Minimal permissions	Chỉ khai báo đúng permissions cần dùng
No remote code	Không load JS từ CDN ngoài — bundle hết vào extension
Privacy policy	Trang `kylenguyen.me` có privacy policy rõ ràng
No data collection	Không có tracking, analytics nào gửi data về server
Single purpose	Extension chỉ làm đúng 1 việc: automate Google Flow
Content Security Policy	Tuân thủ CSP nghiêm ngặt của MV3

XII. 🗺️ Lộ Trình Phát Triển Ước Tính

```
Phase 1 (v1.0): MVP
├── Content Script cơ bản: điền 1 prompt, click generate
├── Side Panel UI đơn giản
└── Manual download

Phase 2 (v1.5): Batch Processing
├── Queue Manager
├── Auto-detect completion
└── Auto-download via chrome.downloads

Phase 3 (v2.0): Pro Features
├── Image-to-Image mode
├── Last frame chaining
├── Concurrency (2 parallel)
└── Progress tracking UI

Phase 4 (v2.3+): Current
├── 6-language support (i18n)
├── Veo 3.1 model support
├── 4K download
├── Character consistency
└── 10,000+ users milestone 🎉
```

XIII. 🧰 Tech Stack Đầy Đủ

Layer	Technology
Extension Standard	Chrome Extension Manifest V3
UI Framework	Vanilla JS + CSS (không cần React/Vue vì extension nhỏ)
Background	Service Worker (ES Modules)
DOM Automation	MutationObserver + Synthetic Events
Data Persistence	`chrome.storage.local`
File Download	`chrome.downloads` API
Internationalization	`chrome.i18n` + `_locales/` folder
Build Tool	Webpack hoặc Vite (để bundle JS)
Size	**711 KiB** (rất tối ưu)

> *[!TIP] **Muốn tự build tương tự?** Bắt đầu từ [Chrome Extension Starter Kit](), học Side Panel API, rồi nghiên cứu MutationObserver để automation DOM của React apps. Phần khó nhất là duy trì CSS selectors khi target website update.*

> *[!NOTE] **Tại sao extension chỉ 711 KiB?** Developer không dùng React/Angular cho Side Panel — Vanilla JS + CSS đủ dùng và giữ kích thước nhỏ gọn. Đây là lựa chọn thông minh cho extension productivity.*